

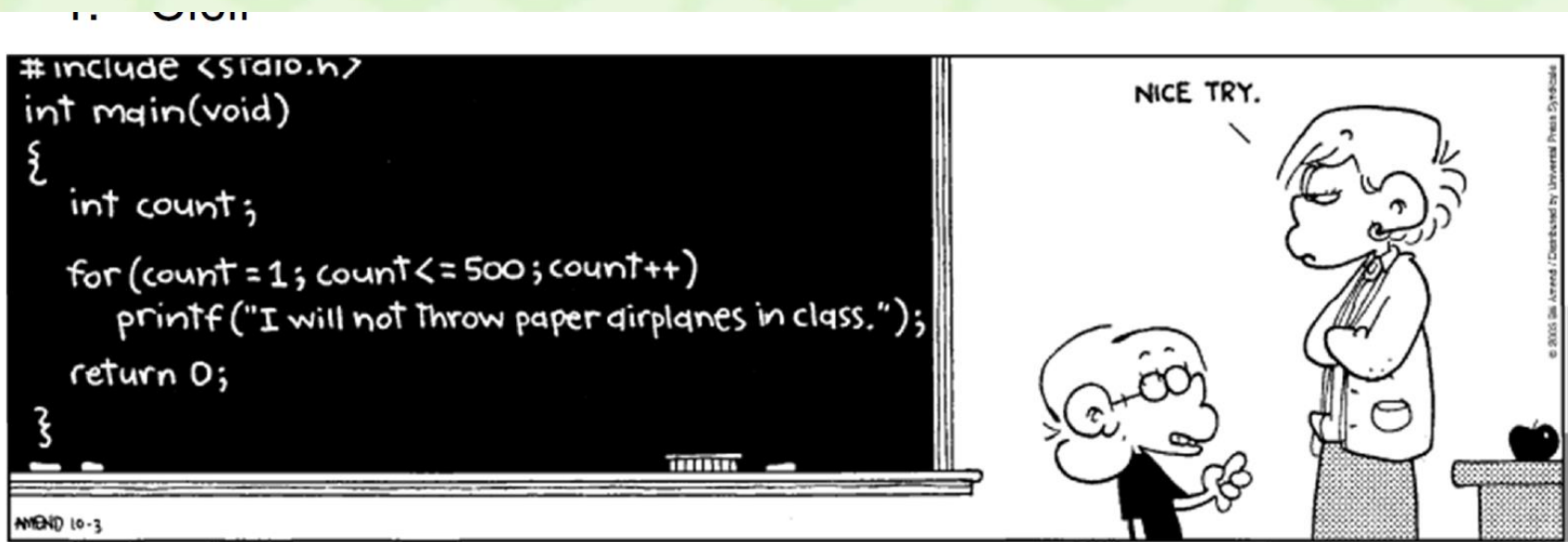
Laboratorio di Programmazione

(Laurea triennale in matematica)

Lezione 8

Abbiamo visto come l'istruzione **for** risulti utile in molte situazioni in cui c'è da ripetere una istruzione (blocco di istruzioni) un numero di volte definito *a priori*.

Vediamo adesso dei casi particolari di utilizzo.



Istruzione **for**

- Sintassi

```
for (<espr1> ; <espr2> ; <espr3> )  
{ <istruzione_1>  
    ...  
    <istruzione_r>  
}  
<istruzione_x>;  
.....
```

Come funziona:

- valuta/ esegue *<espr1>* [inizializzazione]
- valuta (o esegue) *<espr2>*: se ha valore falso procede oltre il ciclo; se ha valore vero, esegue il corpo e poi valuta (o esegue) *<espr3>* [modifica valori di <espr2>]

Se mancano espressioni nel ciclo **for**

- Se manca *espr1* non viene effettuata l'inizializzazione (deve essere effettuata prima del for)
- Se manca *espr2* il ciclo è infinito (attenzione!)
- Se manca *espr3* l'aggiornamento deve essere gestito nel corpo del ciclo

Nota: mancano = sono istruzioni vuote

Esempio: manca *espr1*

```
c=1;  
...  
for (; c<=n; c++) {  
    printf(" * \n");  
}
```

```
for(c=1; c<=n; c++) {  
    printf(" * \n");  
}
```

Equivalente a:



Esempio: manca *espr2*

```
for (c=1;    ;c++) {  
    printf(" * \n");  
}
```

CICLO INFINITO (LOOP)

Esempi: manca *espr3*

```
for(c=1; c<=n;) {  
    printf(" * \n");  
    c++;  
}
```

Equivalente a:

```
for(c=1; c<=n; c++) {  
    printf(" * \n");  
}
```



Operatore virgola

- Combina più espressioni di qualsiasi tipo
 - *esprA, esprB*
- È associativo a sinistra:
 - si valutano le espressioni da sinistra a destra
 - il valore restituito dall'operatore è il valore dell'ultima espressione
 - ha la priorità più bassa (per cui non occorrono parentesi)
- Di solito si usa per inizializzazioni e aggiornamenti multipli nei cicli **for**

Esempi

```
for (a=2, b=1; a<=10 && b<=9; ++a, b=a+b)  
    printf("a=%d    b=%d\n",a,b) ;
```

```
for (a=2, b=1, c=0; a<=10 && c<=20; ++a, c=a+b)  
    printf("a=%d    b=%d    c=%d\n",a,b,c) ;
```

Esempio (senza le istruzioni !)

```
for(c=0; c<=n; c++)  
    printf("%d\n", c+1);
```

...equivalente a:

```
for(c=0; c<=n; printf("%d\n", c+1), c++);
```

Ciclo for "senza istruzioni"

- Può avvenire che non occorranو istruzioni nel corpo di un ciclo perché tutto ha luogo nell'inizializzazione e nell'aggiornamento
 - **for (*espres1*; *espres2*; *espres3*) ;**

Esempio:

```
for (d = 2 ; (n % d != 0) && (d < n) ; d++) ;  
if (d == n) printf("primo") ;
```

Esempio:

```
for (numero=1, somma=0; numero!=0; printf("Inserisci un  
intero:\t) , scanf("%d", &numero) ,  
somma=somma+numero) ;
```

ESEMPIO

```
flag=0;
for(d=2; d<n; d++)
    if (n%d==0) {
        flag=1;
        d=n;
    }
if (flag==0) printf(`'primo'');
```

1

2

```
flag=0;
for(d=2; d<n && flag==0; d++)
    if (n%d==0)
        flag=1;
if (flag==0) printf(`'primo'');
```

```
for (d = 2 ; (n % d != 0) && (d < n ) ; d++) ;
if (d == n) printf("primo");
```

3

Interruzioni (!)

break : interrompe l'esecuzione del **case**, provocando un salto del flusso di esecuzione alla prima istruzione successiva, può forzare la terminazione di un'iterazione **for**, **while** o **do-while**

continue: come **break** impedisce l'esecuzione delle istruzioni successive del ciclo ma provoca una nuova iterazione a partire dall'inizio.

exit: della libreria standard **stdlib.h** provoca l'immediata terminazione del programma e il ritorno al sistema operativo.

goto *<etichetta>*: provoca un salto all'istruzione preceduta da *etichetta* (un identificatore C valido) e dal carattere due punti.

Le ragioni della programmazione strutturata, tra cui pulizia ed eleganza del codice, sconsigliano l'uso generalizzato di **break**, **continue** ed **exit**, di cui vedremo in seguito alcuni esempi di utilizzo e “proibiscono” quello di **goto**.

Istruzione **break**

- In C si può uscire da un ciclo in punti diversi dalla condizione di permanenza (ciò è contrario in linea di principio alla programmazione strutturata)
- L'istruzione **break** provoca l'uscita dal costrutto corrente (**switch**, **while**, **do...while** e **for**)
- È sempre sostituibile modificando la condizione
- Esempio: valuta se n è divisibile per altri numeri oppure è primo

```
for (d = 2 ; d < n ; d++)  
    if (n % d == 0) break;
```

Istruzione **continue**

- L'istruzione **continue** provoca il passaggio all'iterazione successiva, ignorando il resto del corpo (si usa solo nei cicli)
- È sempre sostituibile dalle istruzioni **if** o **if...else**

Esempio

Somma i quadrati dei numeri dispari compresi fra 1 e n

```
s = 0;
for (i = 1 ; i <= n ; i++)
{
    if (i % 2 == 0)
        continue;
    s = s+i*i;
}
```

```
s = 0;
for (i = 1 ; i <= n ; i++)
{
    if (i % 2 != 0)
        s = s+i*i;
}
```

Cicli annidati

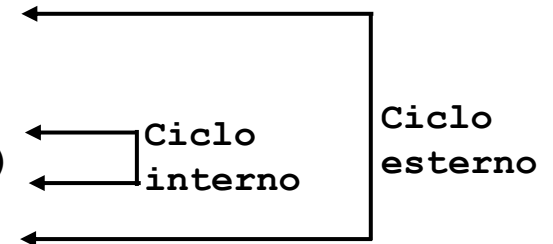
Per ripetere una determinata *istruzione* $n \cdot m$ volte possiamo scrivere:

```
for(i=1; i<=n; i++)  
    for(j=1; j<=m; j++)  
        istruzione;
```

```
#include <stdio.h>
```

```
main()          /* esempio cicli annidati */  
{  
    int n, m, i, j;  
    printf("Inserire numero righe: \t");  
    scanf("%d", &n);  
    printf("Inserire numero colonne: \t");  
    scanf("%d", &m);
```

```
    for(i=1; i<=n; i++) {  
        printf("\n");  
        for(j=1; j<=m; j++)  
            printf("*");  
    }  
}
```



Istruzione break nei cicli annidati

```
#include <stdio.h>

main()      /* esempio cicli annidati */
{
    int n, m, i, j;
    printf("Inserire numero linee: \t");
    scanf("%d", &n);
    printf("Inserire numero colonne: \t");
    scanf("%d", &m);

    for(i=1; i<=n; i++) {
        printf("\n");
        for(j=1; j<=m; j++){
            if (j==3)
                break;
            printf("+");
        }
    }
}
```



Per concludere le strutture di controllo:

Istruzione di selezione multipla: **switch**.

if ...else annidati

```
if(espressione)
    {istruzione}
else
    if (espressione)
        {istruzione}
    else
        if (espressione)
            {istruzione}
        else
            {istruzione}
```

Per scegliere tra diverse situazioni controllate da <espressione> possiamo utilizzare degli **if...else** annidati. Funziona bene.. Solo programma "difficile da leggere".

```
if( mese == 1 )
    printf("Gennaio\n") ;
else if( mese == 2 )
    printf("Febbraio\n") ;
else if( mese == 3 )
    printf("Marzo\n") ;
else if( mese == 4 )
    printf("Aprile\n") ;
else if( mese == 5 )
    printf("Maggio\n") ;
.....
else if( mese == 9 )
    printf("Settembre\n") ;
else if( mese == 10 )
    printf("Ottobre\n") ;
else if( mese == 11 )
    printf("Novembre\n") ;
else if( mese == 12 )
    printf("Dicembre\n") ;
else
    printf("MESE ERRATO!\n") ;
```

Uso
if..else
annidati

```
switch( mese )
{
case 1:
    printf("Gennaio\n") ;
    break ;
case 2:
    printf("Febbraio\n") ;
    break ;
case 3:
    printf("Marzo\n") ;
    break ;
case 4:
    printf("Aprile\n") ;
    break ;
.....
case 12:
    printf("Dicembre\n") ;
    break ;
default:
printf("MESE ERRATO!\n") ;
}
```

Uso
istruzione
switch

Istruzione switch-case

```
switch(espressione) {  
    case costante1:  
        istruzione  
        ...  
        break;  
    case costante2:  
        istruzione  
        ...  
        break;  
    ...  
    [default:  
        istruzione  
        ...    ]  
}
```

Implementa **decisioni multiple** confrontando *espressione* (**int** o **char**) e un insieme di *valori costanti* (ATTENZIONE: solo controllo uguaglianza!).

Anomalie di switch-case

L'istruzione switch è anomala sotto diversi punti di vista:

- Non utilizza le parentesi graffe per l'annidamento delle istruzioni interne
- Prevede solamente il controllo di uguaglianza
- Richiede che i valori da confrontare siano costanti
- **break** e **default** sono opzionali ma utili!

Occorre una forte disciplina nell'utilizzarla correttamente!

```
switch( ora )
{
case <12: NO
    mattino = 1 ;
    break;
case 12 || 20: NO
    pasti = 1 ;
    break;
}
```

NO

OK

```
if (ora<12)
{
    mattino = 1 ;
}
if( ora==12 || ora==20 )
{
    pasti = 1 ;
}
```

```
switch( ora ) {  
  case orapranzo:  
    pranzo = 1 ;  
    break;  
  case oracena:  
    cena = 1 ;  
    break;  
}
```

NO

OK

```
if( ora==orapranzo )  
{  
  pranzo = 1 ;  
}  
else if( ora==oracena )  
{  
  cena = 1 ;  
}
```

switch-case (senza break)

```
switch(espressione) {  
    case costante1:  
        istruzione  
        ...  
    case costante2:  
        istruzione  
        ...  
    case costante3:  
        istruzione  
        ...  
    [default:  
        istruzione  
        ...    ]  
}
```

Verranno eseguite tutte le istruzioni a partire da un case «valido»

```
switch( ora ) {  
case 12:  
    pranzo = 1 ;  
case 20:  
    cena = 1 ;  
}
```

NO

È equivalente a:



```
switch( ora ) {  
case 12:  
    pranzo = 1 ;  
    cena = 1 ;  
    break;  
case 20:  
    cena = 1 ;  
    break;  
}
```

OK

```
switch( ora ) {  
case 12:  
    pranzo = 1 ;  
    break;  
case 20:  
    cena = 1 ;  
    break;  
}
```

switch-case (istruzioni multiple)

```
switch(espressione) {  
    case costante1:  
    case costante2:  
    case costante3:  
        istruzionex  
        ...  
        break;  
    case costante4:  
    case costante5:  
        istruzioney  
        ...  
        break;  
    [default:  
        istruzionez  
        ...    ]  
}
```

Se *espressione* = *costante1-2-3* eseguo *istruzionex* se è
= *costante4-5* eseguo *istruzioney* altrimenti eseguo
istruzionez

```
switch( ora ) {  
case 12:  
    pranzo = 1 ;  
    break;  
case 13:  
    pranzo = 1 ;  
    break;  
case 20:  
    cena = 1 ;  
    break;  
}
```

...è equivalente a:

```
switch( ora ) {  
case 12:  
case 13:  
    pranzo = 1 ;  
    break;  
case 20:  
    cena = 1 ;  
    break;  
}
```

switch-case: esempio 1

```
#include <stdio.h>
int main(void) {
    int x;
    printf("Digita una cifra: ");
    scanf("%d", &x);
    switch(x) {
        case 0:
            printf("zero\n");
            break;
        case 1:
            printf("uno\n");
            break;
        case 2:
            printf("due\n");
            break;
        case 3:
            printf("tre\n");
            break;
        default:
            printf("non compreso\n");
            break;
    }
}
```

switch-case: esempio 2

```
#include <stdio.h>
int main(void) {
    int x;
    printf("Digita una cifra: ");
    scanf("%d", &x);
    switch(x) {
        case 0:
            printf("zero\n");
            //senza istruzione break
        case 1:
            printf("uno\n");
            //senza istruzione break
        case 2:
            printf("due\n");
            //senza istruzione break
        case 3:
            printf("tre\n");
            //senza istruzione break
        default:
            printf("non compreso\n");
    }
}
```

switch-case: esempio 3

```
#include <stdio.h>
int main(void){
    char x;
    printf("Digita una cifra: ");
    scanf("%c", &x);
    switch(x) {
        case '2':
        case '4':
        case '6':
            printf("pari\n");
            break;
        case '1':
        case '3':
        case '5':
            printf("dispari\n");
            break;
        default:
            printf("altro\n");
    }
}
```